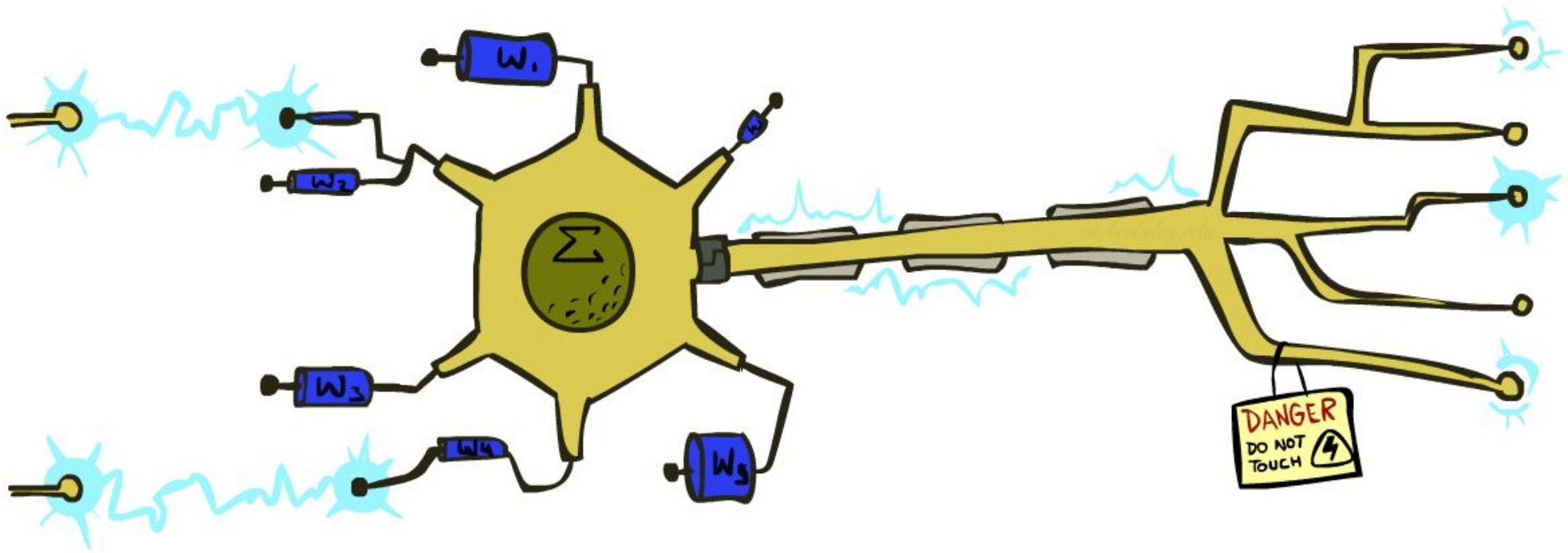
CS 188: Artificial Intelligence

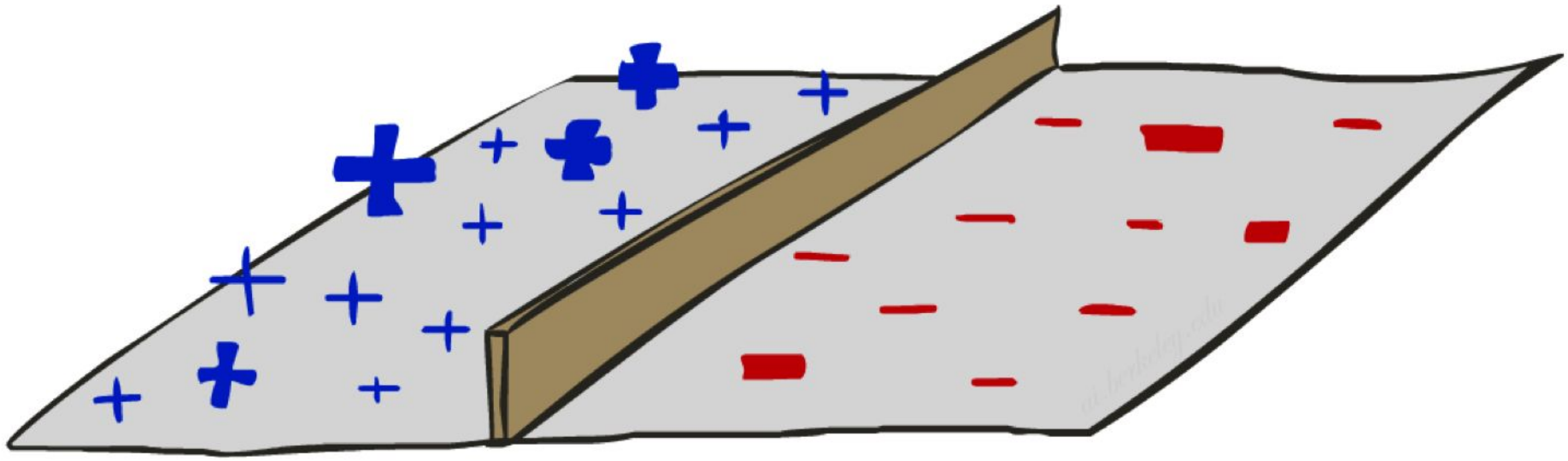
Deep Learning



Instructors: Dan Klein and Stuart Russell

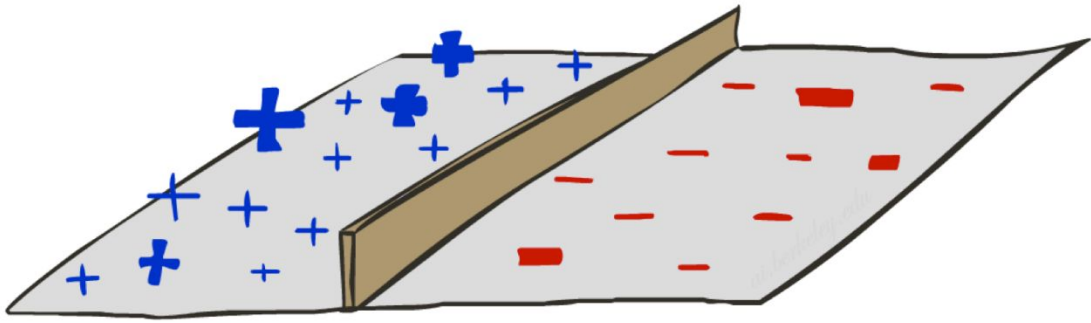
University of California, Berkeley

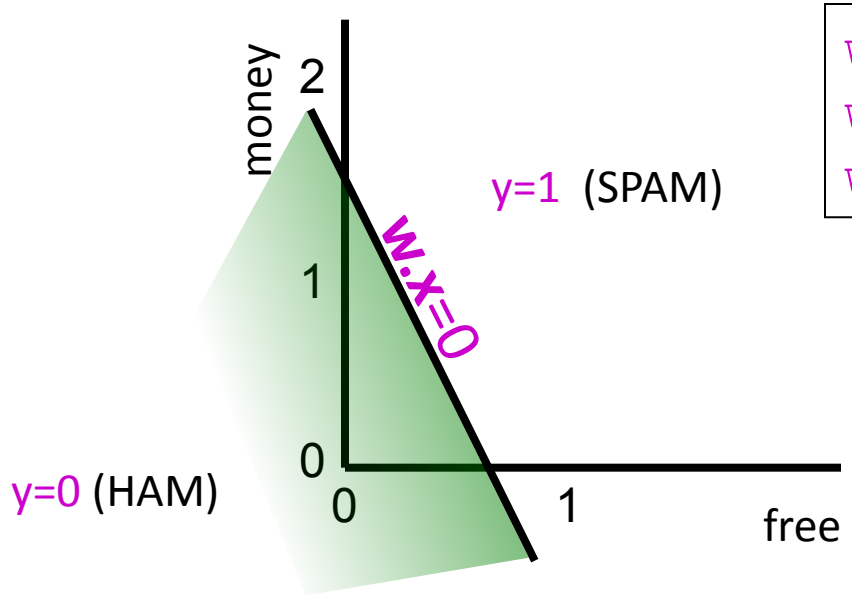
Threshold perceptron as linear classifier



Binary Decision Rule

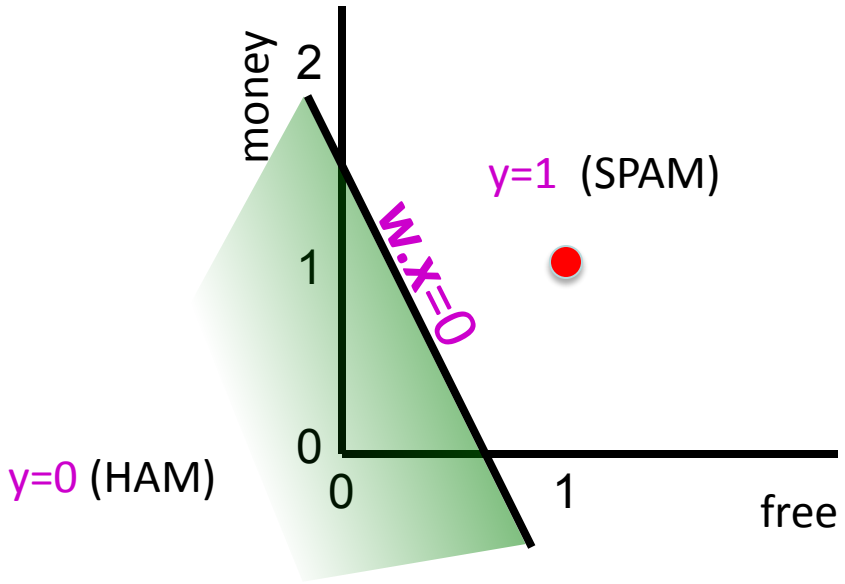
- A ***threshold perceptron*** is a single unit that outputs
  - $y = h_{\mathbf{w}}(\mathbf{x}) = 1$ when $\mathbf{w}.\mathbf{x} \geq 0$
    $= 0$ when $\mathbf{w}.\mathbf{x} < 0$
- In the input vector space
  - Examples are points **x**
  - The equation **w.x**=0 defines a ***hyperplane***
  - One side corresponds to y=1
  - Other corresponds to y=0





w_0	:	−3
w_{free}	:	4
w_{money}	:	2

Example



w_0	:	-3
w_{free}	:	4
w_{money}	:	2

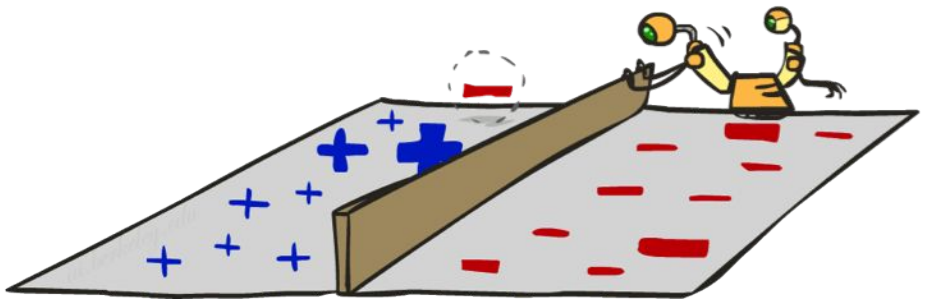
x_0	:	1
x_{free}	:	1
x_{money}	:	1

Dear Stuart, I'm leaving Macrosoft to return to academia. The **money** is is great here but I prefer to be **free** to do my own research; and I *really* love teaching undergrads!
Do I need to finish my BA first before applying?
Best wishes
Bill

$$\mathbf{w.x} = -3x1 + 4x1 + 2x1 = 3$$

Perceptron learning rule

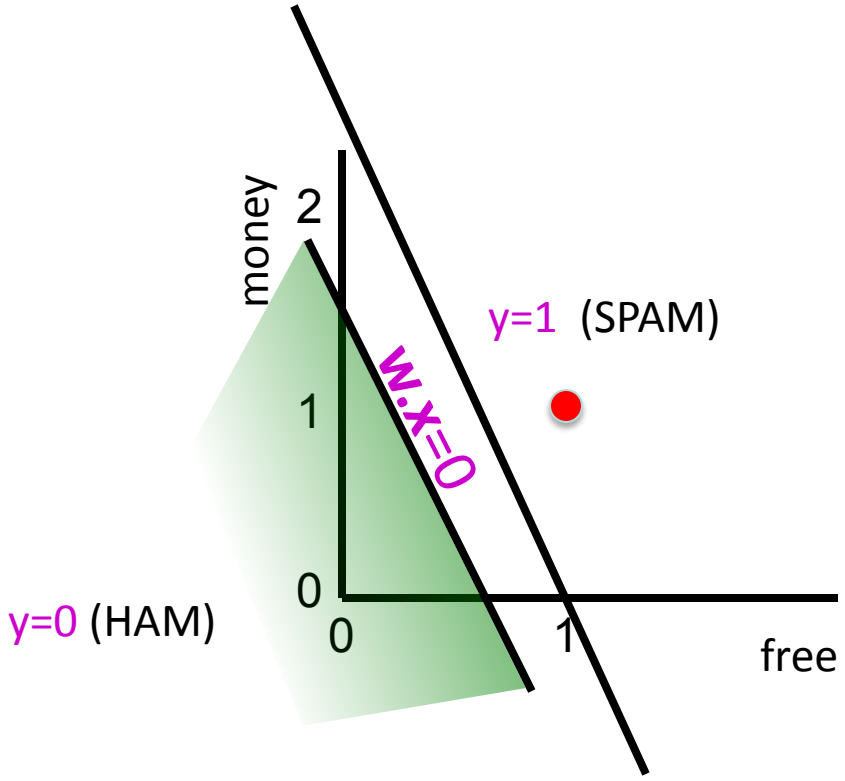
- If true $y \neq h_{\mathbf{w}}(\mathbf{x})$ (an error), adjust the weights
- If $\mathbf{w}.\mathbf{x} < 0$ but the output should be $y=1$
  - This is called a ***false negative***
  - Should ***increase*** weights on ***positive*** inputs
  - Should ***decrease*** weights on ***negative*** inputs
- If $\mathbf{w}.\mathbf{x} > 0$ but the output should be $y=0$
  - This is called a ***false positive***
  - Should ***decrease*** weights on ***positive*** inputs
  - Should ***increase*** weights on ***negative*** inputs
- The ***perceptron learning rule*** does this:
  - $\mathbf{w} \leftarrow \mathbf{w} + \alpha\,(y - h_{\mathbf{w}}(\mathbf{x}))\,\mathbf{x}$



learning rate

+1, -1, or 0 (no error)

Example



w_0	:	-3
w_{free}	:	4
w_{money}	:	2

x_0	:	1
x_{free}	:	1
x_{money}	:	1

Dear Stuart, I wanted to let you know that I have decided to leave Macrosoft and return to academia. The **money** is is great here but I prefer to be **free** to pursue more interesting research and I *really* love teaching undergraduates! Do I need to finish my BA first before applying?
Best wishes
Bill

$\mathbf{w.x} = -3x1 + 4x1 + 2x1 = 3$

$\mathbf{w} \leftarrow \mathbf{w} + \alpha\,(y - h_{\mathbf{w}}(\mathbf{x}))\,\mathbf{x}$

$\alpha = 0.5$

$\mathbf{w} \leftarrow (-3,4,2) + 0.5\,(0 - 1)\,(1,1,1)$

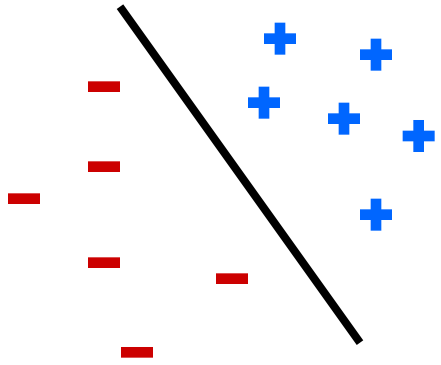
$= (-3.5,3.5,1.5)$

Perceptron convergence theorem

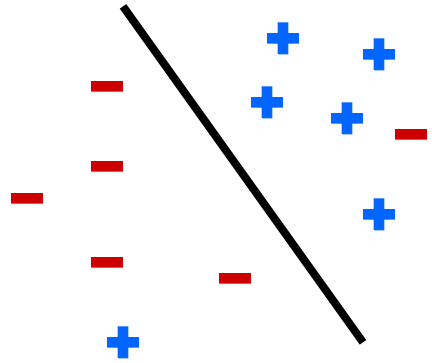
- A learning problem is ***linearly separable*** iff there is some hyperplane exactly separating +ve from –ve examples

- Convergence: if the training data are ***separable***, perceptron learning applied repeatedly to the training set will eventually converge to a perfect separator

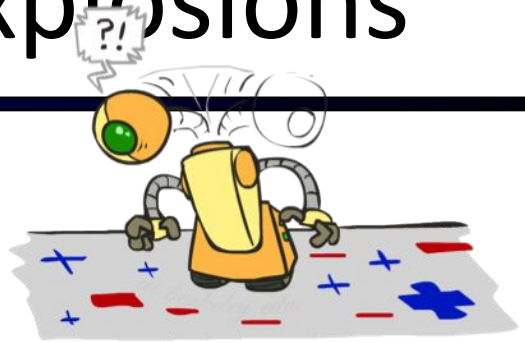
Separable



Non-Separable



Example: Earthquakes vs nuclear explosions

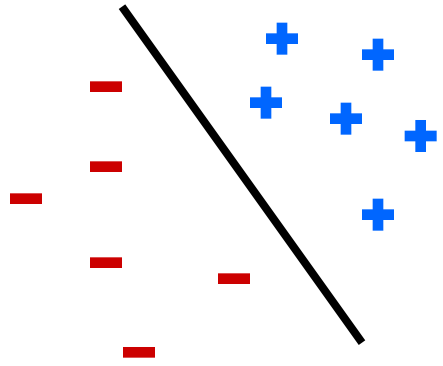


63 examples, 657 updates required

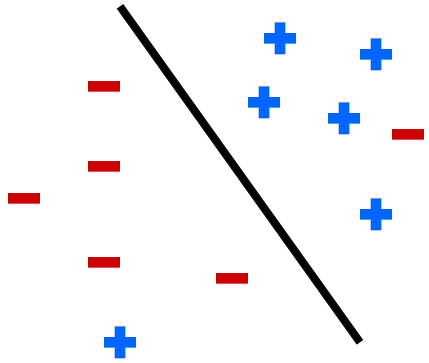
Perceptron convergence theorem

- A learning problem is ***linearly separable*** iff there is some hyperplane exactly separating +ve from –ve examples
- Convergence: if the training data are separable, perceptron learning applied repeatedly to the training set will eventually converge to a perfect separator
- Convergence: if the training data are ***non-separable***, perceptron learning will converge to a minimum-error solution provided the learning rate α is decayed appropriately (e.g., α=1/t)

Separable



Non-Separable



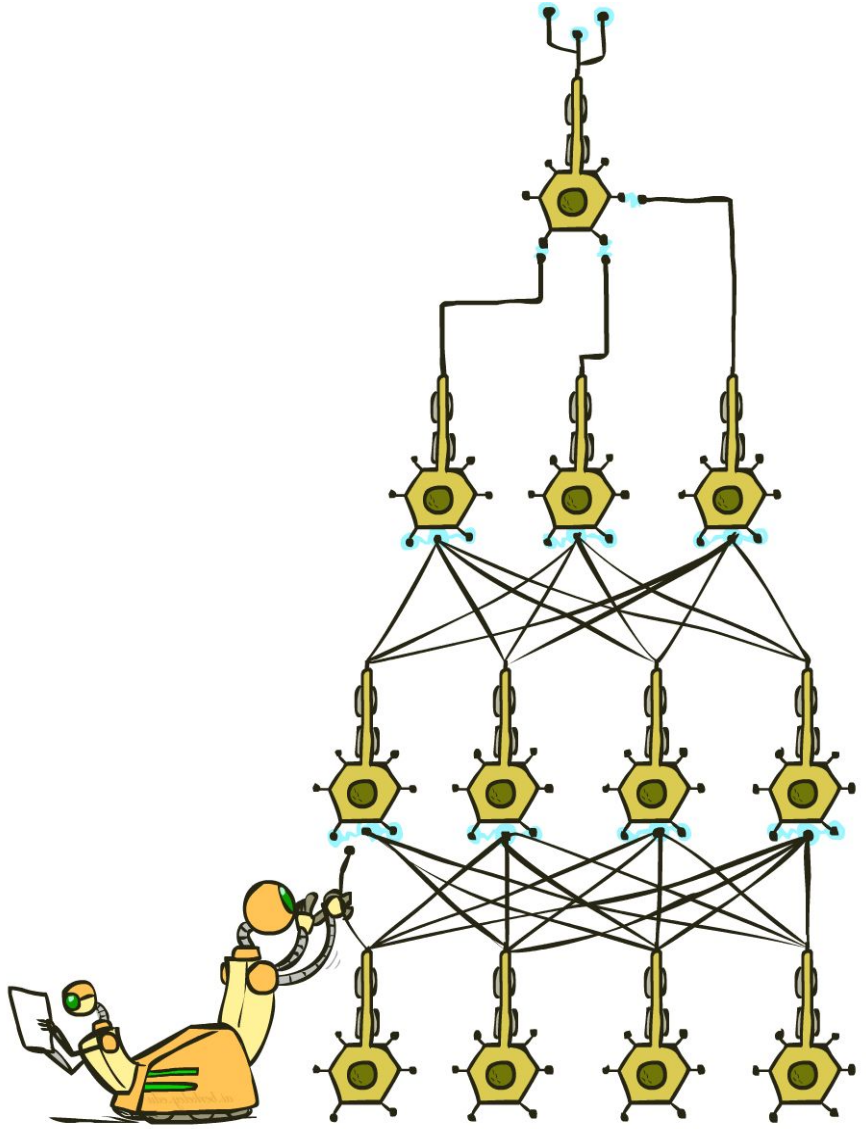
Perceptron learning with fixed α

71 examples, 100,000 updates
fixed $\alpha = 0.2$, no convergence

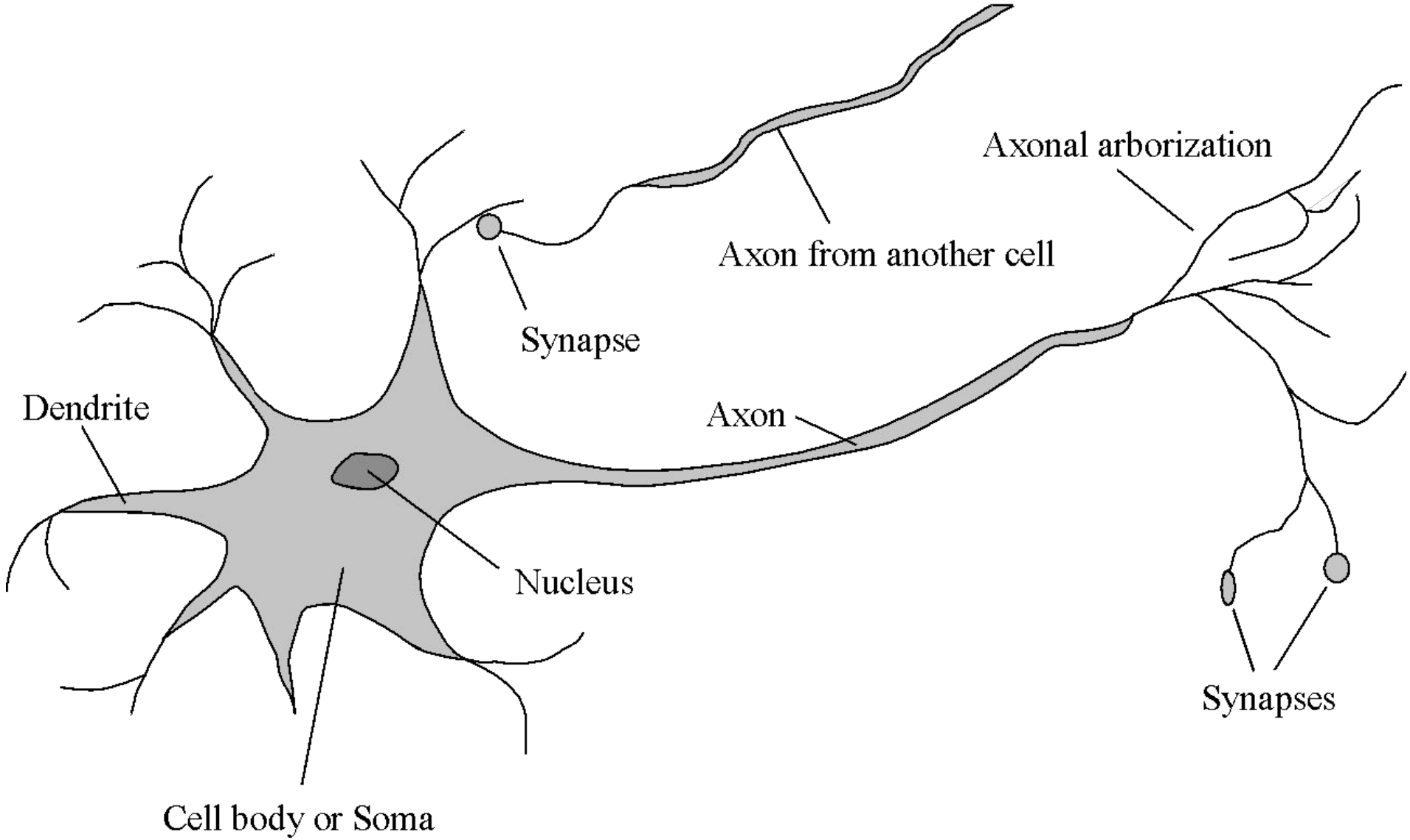
Perceptron learning with decaying α

71 examples, 100,000 updates
decaying $\alpha = 1000/(1000 + t)$, near-convergence

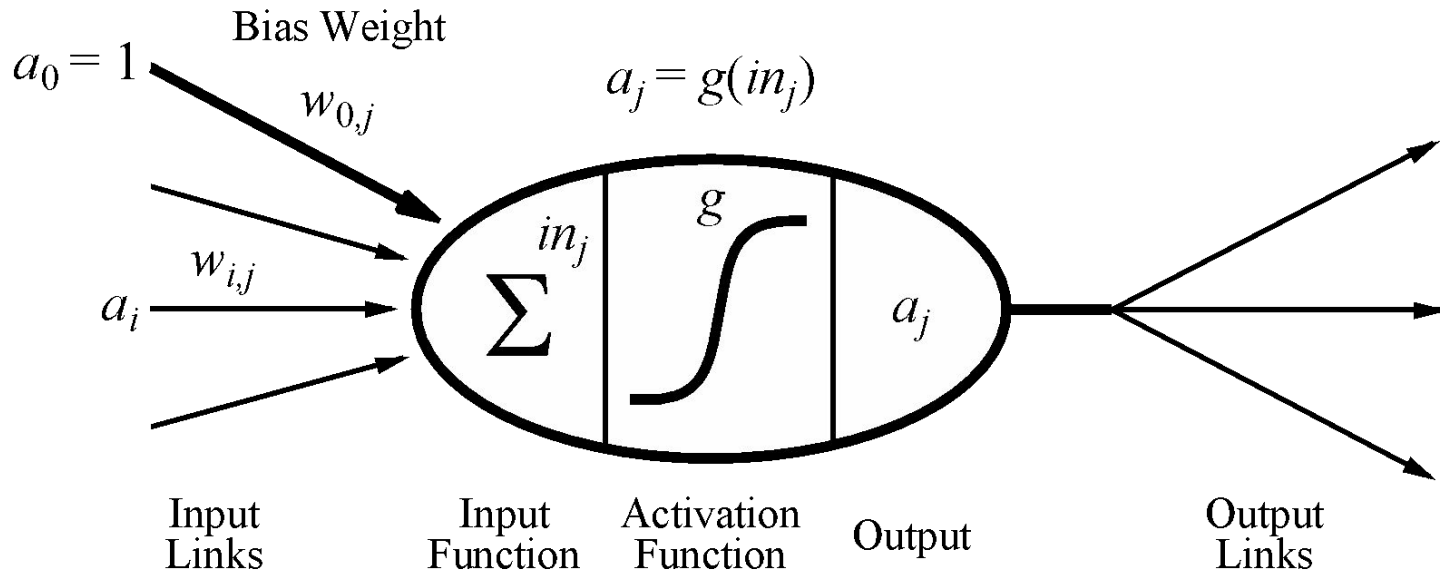
Neural Networks



Very Loose Inspiration: Human Neurons



Simple Model of a Neuron (McCulloch & Pitts, 1943)



- Inputs a_i come from the output of node i to this node j (or from "outside")
- Each input link has a ***weight*** $w_{i,j}$
- There is an additional fixed input $a_0=1$ with ***bias*** weight $w_{0,j}$
- The total input is $in_j = \Sigma_i w_{i,j} a_i$
- The output is $a_j = g(in_j) = g(\Sigma_i w_{i,j} a_i) = g(\mathbf{w.a})$

Activation Functions

Sigmoid:

$$g(x) = \frac{1}{1 + e^{-x}}$$

ReLU:

$$g(x) = max(0, x)$$

Softplus:

$$g(x) = log(1 + e^{x})$$

Tanh:

$$g(x) = \frac{e^{2x} + 1}{e^{2x} - 1}$$

Training a Single Neuron

- Data point $\boldsymbol{x}$, label y
- Define loss (e.g., squared error loss L2)
  - $Loss(\boldsymbol{w}) = (y - h_{\boldsymbol{w}}(\boldsymbol{x}))^2$
    $= (y - g(\boldsymbol{w}.\boldsymbol{x}))^2$
- Step each weight along (negative) derivative of loss function
  - Gradient descent $w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} Loss(\boldsymbol{w})$

Reminder: Chain rule

- $\frac{d}{dx} f\big(g(x)\big) = f'\big(g(x)\big) \frac{d}{dx} g(x)$
- $\frac{\partial}{\partial w_i} Loss(\boldsymbol{w}) = \frac{\partial}{\partial w_i} (y - g(\boldsymbol{w}.\boldsymbol{x}))^2$

$$= 2\,(y - g(\boldsymbol{w}.\boldsymbol{x}))\, \frac{\partial}{\partial w_i} (y - g(\boldsymbol{w}.\boldsymbol{x}))$$

$$= -2\,(y - g(\boldsymbol{w}.\boldsymbol{x}))\, \frac{\partial}{\partial w_i} g(\boldsymbol{w}.\boldsymbol{x})$$

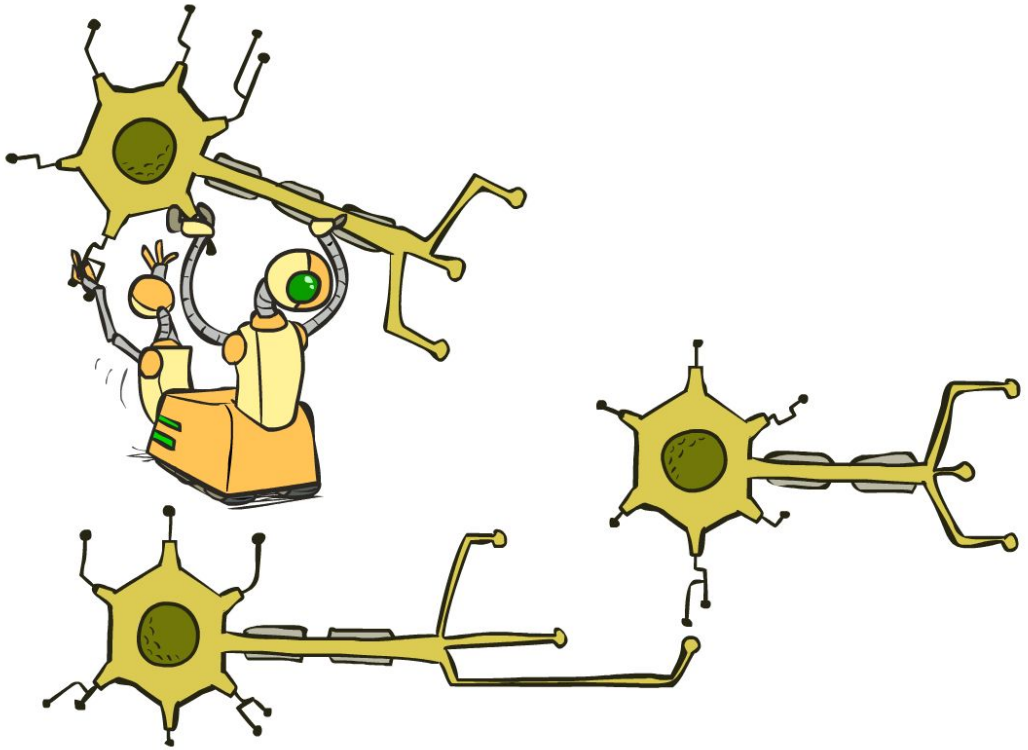
$$= -2\,(y - g(\boldsymbol{w}.\boldsymbol{x}))\, g'(\boldsymbol{w}.\boldsymbol{x})\, \frac{\partial}{\partial w_i} (\boldsymbol{w}.\boldsymbol{x})$$

Practical Improvements to Gradient Descent

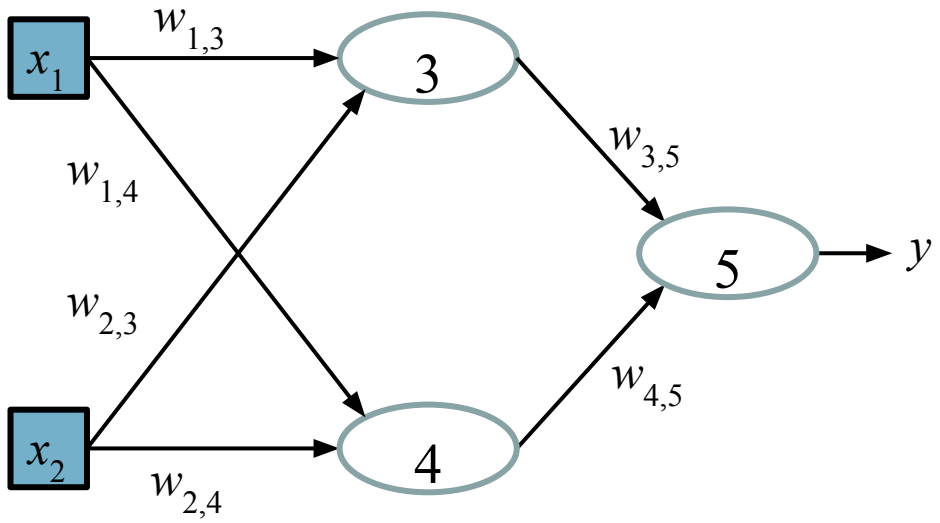
- Standard gradient descent:
  - Compute gradient w.r.t. all training examples at once, take a step
- Stochastic gradient descent:
  - Pick a random training example, compute gradient, take a step
- Stochastic gradient descent with mini-batches:
  - Pick *k* random training examples, compute gradient, take a step
  - Choose *k* to fit the GPU architecture

Multilayer Perceptrons

- A ***multilayer perceptron (MLP)*** is a feedforward network with at least one ***hidden layer*** (nodes that are neither inputs nor outputs)
- MLPs with enough hidden nodes can represent any function



Neural Networks Are Just Functions



$$h_{\boldsymbol{w}}(x) = g_5\begin{pmatrix} w_{0,5} + w_{3,5}g_3(w_{0,3} + w_{1,3}x_1 + w_{2,3}x_2) \\ + w_{4,5}g_4(w_{0,4} + w_{1,4}x_1 + w_{2,4}x_2) \end{pmatrix}$$

Computation graph

Error Back-propagation

- Training proceeds exactly as before:
  - $w_{i,j} \leftarrow w_{i,j} - \alpha \frac{\partial}{\partial w_{i,j}} Loss(\boldsymbol{w})$
  - Apply chain rule (a lot!!)
- ***Error back-propagation***: we can compute all the derivatives in a single backward pass through the network
  - Works for any acyclic computation graph!
  - So we can try many different kinds of network architectures, activation functions, etc., without doing any new math or writing any new code!!!!

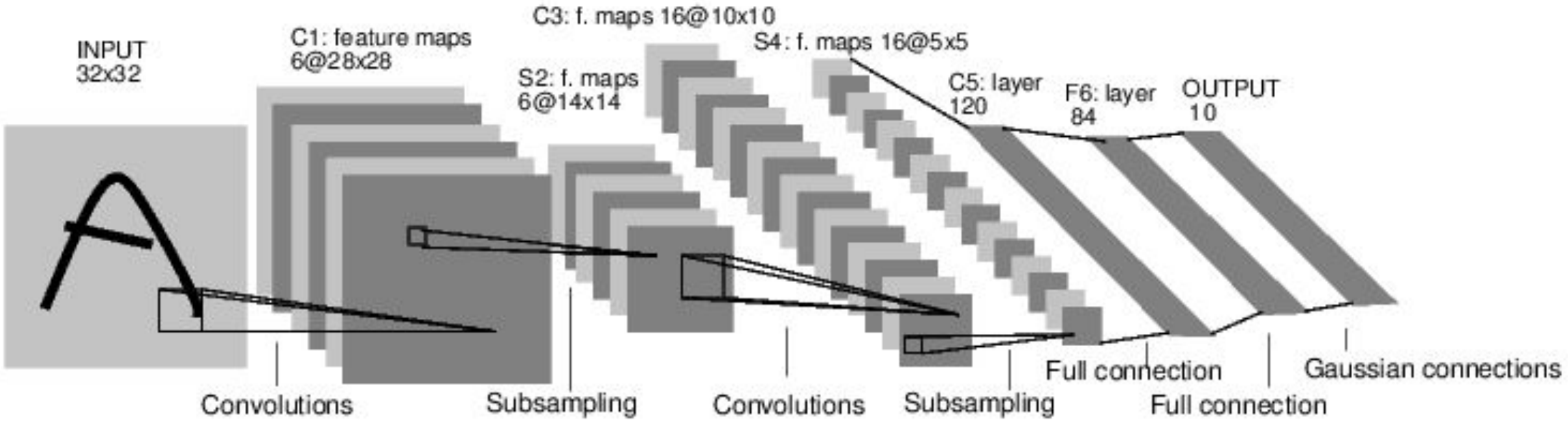
Error Back-propagation contd.

- Let f, g, h, j, k be arbitrary functions
- Define "backward message" $\frac{\partial L}{\partial h_j}$ to be the partial derivative of L with respect to the input to j coming from h
  - This input is computed in the forward pass
- Note that $\frac{\partial L}{\partial h} = \frac{\partial L}{\partial h_j} + \frac{\partial L}{\partial h_k}$
- $\frac{\partial L}{\partial f_h} = \frac{\partial L}{\partial h}\frac{\partial h}{\partial f_h}$ and $\frac{\partial L}{\partial g_h} = \frac{\partial L}{\partial h}\frac{\partial h}{\partial g_h}$
- $\frac{\partial h}{\partial f_h}$ and $\frac{\partial h}{\partial g_h}$ are derivatives of h with respect to its inputs

Backward pass reaches each weight node w where the incoming messages sum to $\partial L / \partial w$

Convolutional Neural Networks

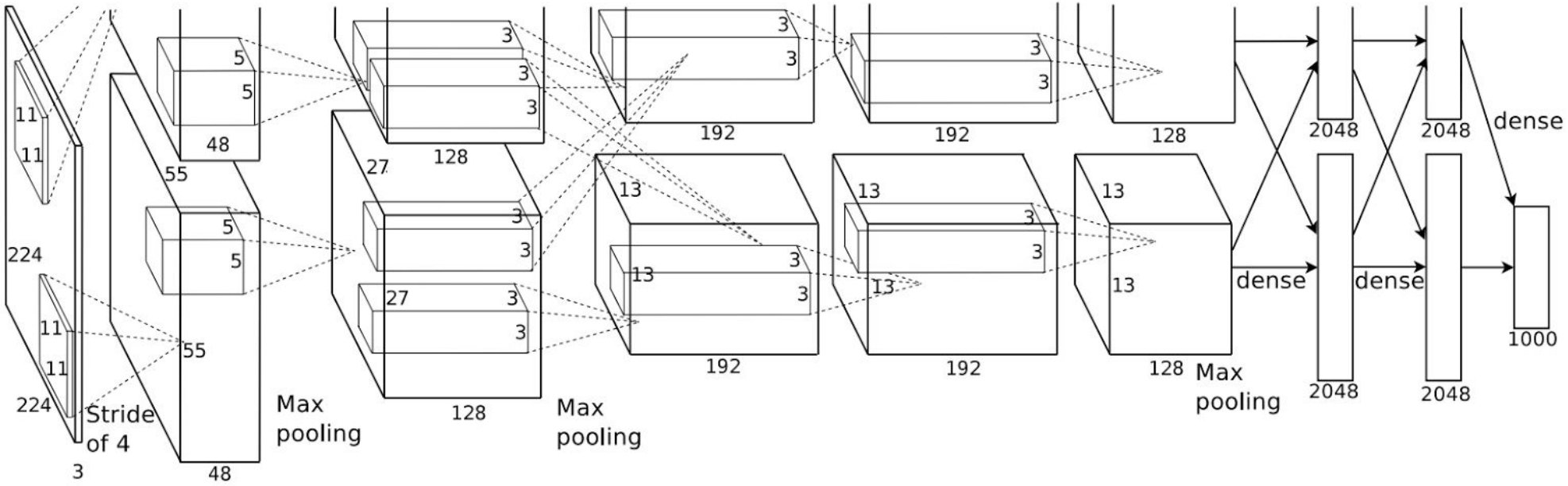
- LeNet5 (Lecun, et al, 1998): CNNs for digit recognition



LeCun, Yann, et al. "Gradient-based learning applied to document recognition." Proceedings of the IEEE 86.11 (1998): 2278-2324.

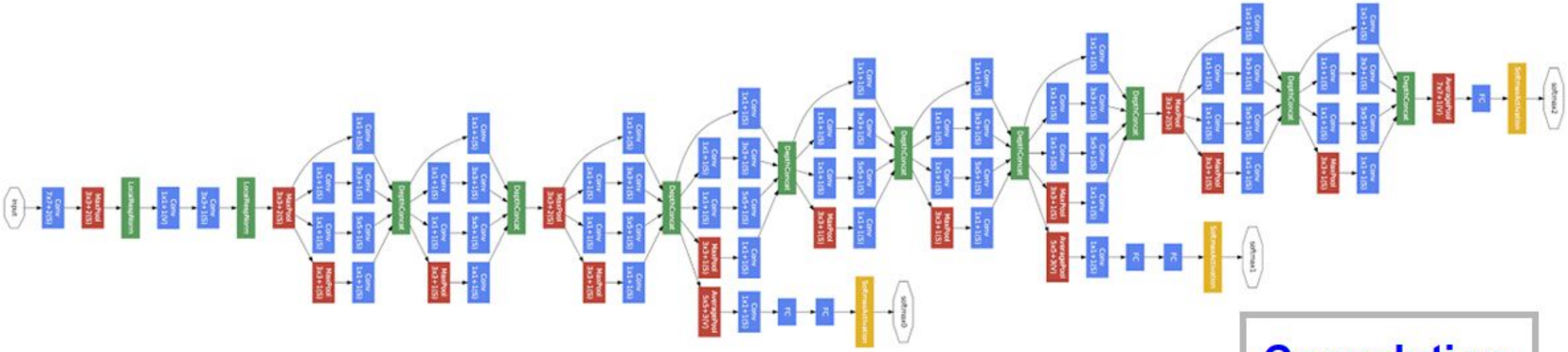
Convolutional Neural Networks

- Alexnet – Alex Krizhevsky, Geoffrey Hinton, et al, 2012
  - CNNs for image classification
  - More data & more compute power (60 million parameters)



Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." Advances in neural information processing systems. 2012.

Deep Learning: GoogLeNet



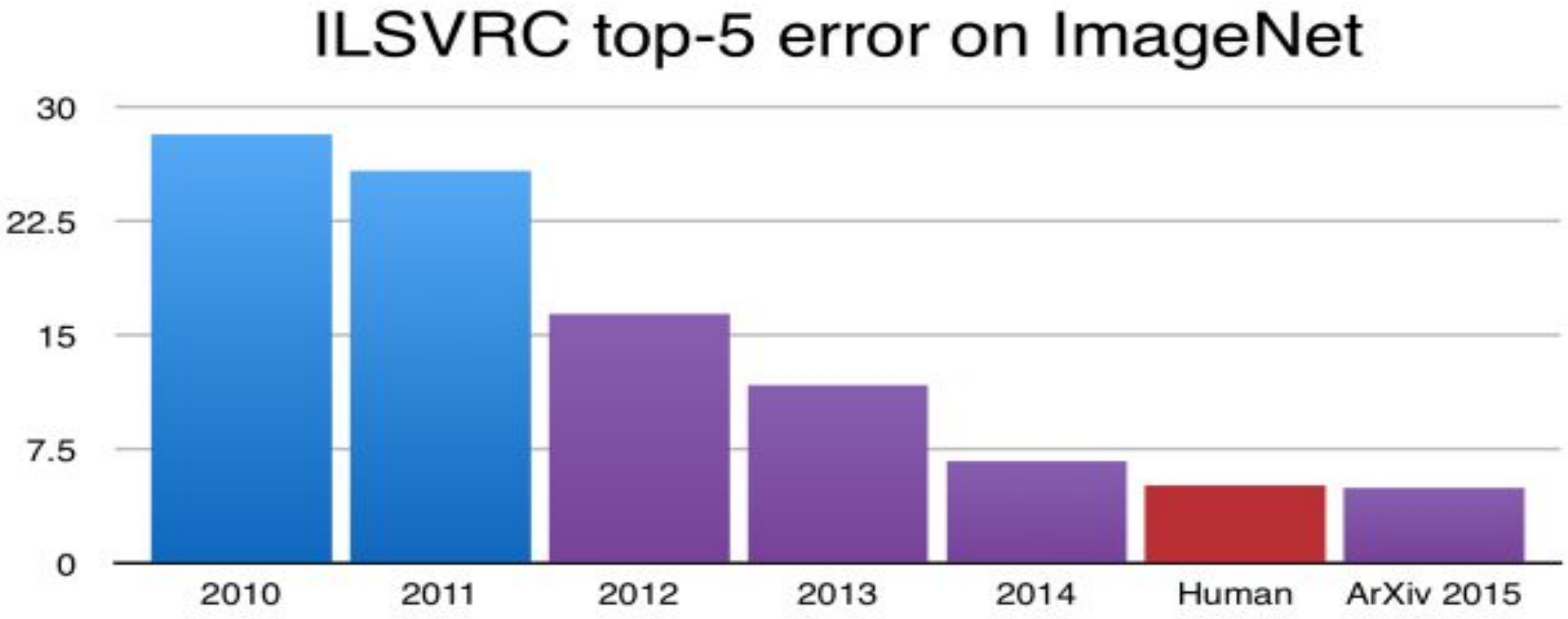
Convolution
Pooling
Softmax
Other

Szegedy, Christian, et al. ”Going deeper with convolutions." CVPR (2015).

ImageNet (14M images, 20K categories)

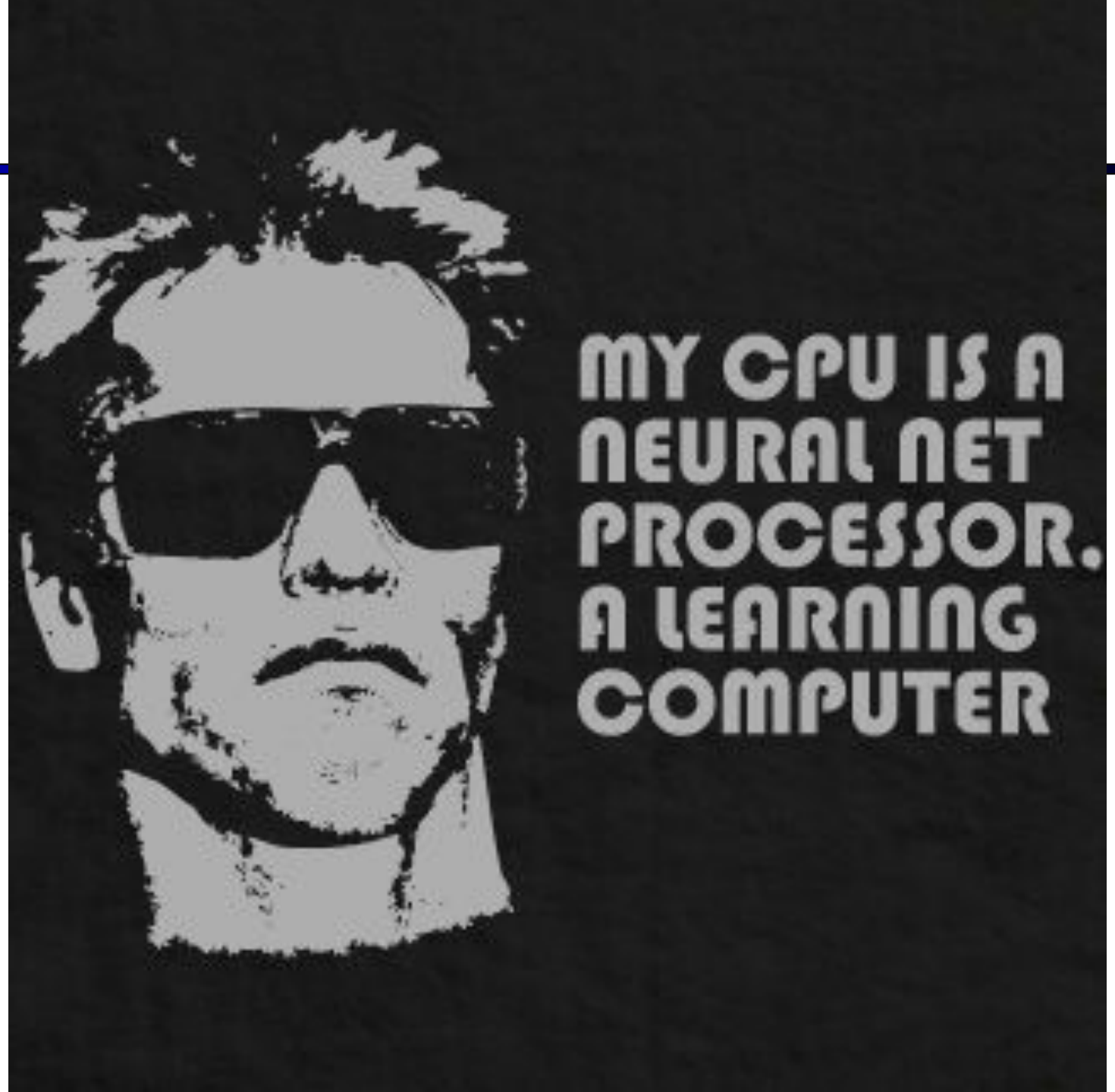
Recognition performance

ILSVRC competition: 1000 categories



AARIAN MARSHALL TRANSPORTATION 09.13.17
07:00 AM

TESLA BEARS SOME BLAME FOR SELF-DRIVING CRASH DEATH, FEDS SAY


MY CPU IS A
NEURAL NET
PROCESSOR.
A LEARNING
COMPUTER

Practical Issues